

ESP32 Digital Signature Micro-Benchmark

Phase A: Isolated Cryptographic Evaluation (No Network Overhead)

FYP Documentation: Embedded VPN Client (Deliverable 2)

Department of Computer Engineering

October 2026

1 Phase A: Isolated Micro-Benchmarks (No Network Overhead)

Phase A evaluates the isolated, raw cryptographic execution of three digital signature algorithms directly on an ESP32 microcontroller at an equivalent 128-bit classical security baseline. All tests execute entirely within local static and dynamic RAM without Wi-Fi or TCP transport latency to establish baseline hardware performance, memory consumption, and frame overhead:

1. **Ed25519:** Modern Twisted Edwards Curve25519 digital signature algorithm implemented via the optimized Libsodium cryptographic library.
2. **ECDSA P-256:** Standard NIST prime curve `secp256r1` digital signature scheme implemented via ESP32 Mbed TLS.
3. **Classical RSA-3072:** Classical Rivest-Shamir-Adleman algorithm configured with a 3072-bit modulus, PKCS#1 v1.5 padding, and SHA-256 implemented via ESP32 Mbed TLS.

1.1 Microcontroller Test Environment

- **Microcontroller Hardware:** Espressif ESP32-D0WDQ6 (Revision v1.0), Dual-Core Xtensa LX6.
- **Processor Clock Frequency:** 240 MHz (240,000,000 cycles/second).
- **Available Static/Dynamic SRAM:** ~ 329 KB free heap at benchmark startup.
- **Cycle-Accurate Measurement:** Read directly from the Xtensa hardware 32-bit cycle register (`ccount`). Because the clock operates at a fixed 240 MHz:

$$\text{Latency (ms)} = \frac{\text{Hardware CPU Cycles}}{240,000} \quad (1)$$

2 Key Performance Indicators (KPIs)

The evaluation profiles seven Key Performance Indicators (KPIs) covering execution latency, hardware workload, wire payload footprint, and dynamic RAM consumption:

2.1 KPI Definitions and Operational Impact

- **SIG-1 (Key Generation Latency):** Hardware cycles and elapsed milliseconds required to generate a fresh asymmetric keypair (public and private key). Crucial if ephemeral client authentication keys are generated per VPN session.
- **SIG-2 (Public Key Wire Size):** Size in bytes of the serialized public key exchanged during mutual identity establishment.
- **SIG-3 (Message Signing Latency):** Hardware cycles and elapsed milliseconds to sign a 64-byte cryptographic authentication challenge. Directly determines client response latency.
- **SIG-4 (Signature Wire Size):** Byte length of the generated signature transmitted over the network in authentication response frames.

- **SIG-5 (Verification Latency):** Hardware cycles and elapsed milliseconds to verify the peer’s digital signature on a 64-byte challenge. Governs gateway authentication processing time.
- **SIG-6 (Total Client Sign Time):** Total on-chip processor execution latency (SIG-1+SIG-3) required for complete client-side key setup and challenge signing.
- **SIG-7 (Peak Dynamic Heap Memory):** Maximum dynamic RAM allocated from the ESP32 heap during cryptographic execution. Minimizing dynamic allocation is critical to prevent heap fragmentation and Out-Of-Memory (OOM) crashes in memory-constrained IoT nodes.

2.2 Iteration Count and Statistical Reliability Strategy

- **Fast Benchmark Iterations ($N = 100$):** For Ed25519, ECDSA P-256, and fast RSA signing/verification, 100 independent iterations were measured to compute stable arithmetic means and eliminate OS scheduling artifacts.
- **RSA Key Generation Iterations ($N = 10$):** Because 3072-bit RSA key generation requires stochastic prime searching across thousands of Miller-Rabin tests (taking ~ 9.6 seconds per key), 10 iterations were executed with watchdog yield intervals (`vTaskDelay(1)`) to avoid triggering the ESP32 Task Watchdog Timer (TWDT).

Table 1: Summary of Digital Signature Benchmarking KPIs

KPI ID	Metric Name	Unit	Operational Significance for Embedded VPN
SIG-1	Key Generation Time	ms / cycles	Ephemeral identity generation speed
SIG-2	Public Key Wire Size	Bytes	Bandwidth overhead of client identity advertisement
SIG-3	Message Signing Time	ms / cycles	Time required to sign challenge nonce during handshake
SIG-4	Signature Wire Size	Bytes	Bandwidth overhead of authentication challenge proof
SIG-5	Verification Time	ms / cycles	Peer verification latency
SIG-6	Total Client Sign Time	ms / cycles	Complete client-side crypto latency (SIG-1+SIG-3)
SIG-7	Peak Dynamic Heap RAM	Bytes	Peak RAM allocated from heap during operation

3 Benchmark Results and Comparison

3.1 Empirical Performance Comparison

Table 2 presents the empirical measurements captured directly from the ESP32 serial console across all three signature schemes at 240 MHz.

4 Selection Rationale for Ed25519

Ed25519 is unequivocally selected as the primary mutual authentication and digital signature algorithm for the embedded VPN client based on four critical engineering justifications:

1. **Superior Client Execution Speed ($17\times$ faster than ECDSA P-256):** Ed25519 completes key generation in **8.56 ms** and challenge signing in **9.42 ms**, resulting in a total client cryptographic workload of only **17.99 ms**. In contrast, ECDSA P-256 requires **305.51 ms** ($17.0\times$ slower) and RSA-3072 requires **10,006.61 ms** (≈ 10 seconds, $556\times$ slower). Completing client-side authentication in under 20 ms minimizes device active state and preserves battery longevity.
2. **Minimal Transmission Wire Overhead ($8\times$ smaller than RSA-3072):** Ed25519 features extremely compact cryptographic representations: 32 bytes for the public key and 64

Table 2: Performance Comparison: Ed25519 vs. ECDSA P-256 vs. Classical RSA-3072

Metric	Ed25519	ECDSA P-256	RSA-3072	Winner	Performance Factor
Execution Latency (Milliseconds)					
Key Generation Time (SIG-1)	8.56 ms	146.93 ms	9,630.80 ms	Ed25519	17.2× faster than P-256
Message Signing Time (SIG-3)	9.42 ms	158.58 ms	375.81 ms	Ed25519	16.8× faster than P-256
Verification Time (SIG-5)	19.80 ms	313.88 ms	1.71 ms	RSA-3072	15.9× faster than P-256
Total Client Sign Time (SIG-6)	17.99 ms	305.51 ms	10,006.61 ms	Ed25519	17.0× faster than P-256
Hardware CPU Workload (Cycles @ 240 MHz)					
KeyGen Cycles	2,055,308	35,262,354	2,311,393,015	Ed25519	Lowest compute cycles
Signing Cycles	2,261,378	38,059,479	90,194,354	Ed25519	Lowest compute cycles
Verification Cycles	4,750,949	75,331,369	410,321	RSA-3072	Shortest verification
Total Client Cycles	4,316,686	73,321,833	2,401,587,369	Ed25519	Lowest total workload
Communication Wire Footprint (Bytes)					
Public Key Wire Size (SIG-2)	32 bytes	65 bytes	384 bytes	Ed25519	12.0× smaller than RSA
Signature Wire Size (SIG-4)	64 bytes	71 bytes	384 bytes	Ed25519	6.0× smaller than RSA
Total Wire Size (Key + Sig)	96 bytes	136 bytes	768 bytes	Ed25519	8.0× smaller than RSA
Memory Utilization					
Peak Dynamic Heap (SIG-7)	108 bytes	300 bytes	2,068 bytes	Ed25519	19.1× less heap than RSA

bytes for the signature, yielding an aggregate wire overhead of just **96 bytes**. ECDSA requires 136 bytes (using 65-byte uncompressed points and 71-byte ASN.1 DER signatures), while RSA-3072 requires **768 bytes**. The compact 96-byte payload avoids TCP packet segmentation and reduces Wi-Fi radio transmission time.

3. **Ultra-Low Dynamic RAM Consumption (108 bytes vs. 2,068 bytes):** The ESP32 allocates only **108 bytes** of dynamic heap memory during Ed25519 signature operations via Libsodium. ECDSA P-256 allocates 300 bytes for its point multiplication context, while RSA-3072 allocates **2,068 bytes** to maintain multi-precision integer (MPI) BigInt arithmetic structures. Ed25519 prevents heap fragmentation across prolonged uptime.
4. **Constant-Time Execution and Immunity to Side-Channel Attacks:** Ed25519 operations over Twisted Edwards Curve25519 are inherently constant-time, eliminating timing and cache side-channel vulnerabilities. In contrast, standard ECDSA implementations require high-entropy per-signature secret nonces (k); any bias or leakage in k exposes the private key. Ed25519 derives its nonce deterministically ($r = H(\text{hash_prefix} \parallel M)$), completely eliminating catastrophic nonce collisions.
5. **Impracticality of Classical RSA-3072 on Embedded Clients:** While RSA-3072 achieves rapid verification (**1.71 ms**) due to its small public exponent ($e = 65537$), this benefit is entirely asymmetric and serves only the verifying gateway. On the embedded client side, RSA-3072 suffers from an unviable ~ 9.6 -second key generation time, 375.81 ms signing latency, 768-byte wire overhead, and 2 KB heap allocation, rendering it inappropriate for resource-constrained IoT endpoints.

5 Serial Monitor Output Verification

The serial captures below verify the empirical test executions recorded directly on the ESP32 hardware:

5.1 Ed25519 Benchmark Execution Capture

Figure 1 displays the serial monitor output for Ed25519 over 100 runs, demonstrating 8.56 ms keygen, 9.42 ms signing, 19.80 ms verification, and 108 bytes peak heap RAM.

```

=====
ESP32 Digital Signature Micro-Benchmark (Phase A: Isolated)
Fast Runs: 100 | RSA KeyGen Runs: 10 | CPU: 240 MHz | Heap: 329392 bytes
=====

[1/3] Ed25519 (Curve25519)
SIG-1 Key Generation Latency : 2055308 cycles | 8.56 ms
SIG-2 Public Key Wire Size : 32 bytes
SIG-3 Message Signing Latency : 2261378 cycles | 9.42 ms
SIG-4 Signature Wire Size : 64 bytes
SIG-5 Verification Latency : 4750949 cycles | 19.80 ms
SIG-6 Total Client Sign Time : 4316686 cycles | 17.99 ms
SIG-7 Peak Dynamic Heap Memory : 108 bytes

```

Figure 1: ESP32 Serial Output for Ed25519 micro-benchmarks (100 runs @ 240 MHz).

5.2 ECDSA P-256 Benchmark Execution Capture

Figure 2 displays the serial monitor output for ECDSA P-256 over 100 runs, recording 146.93 ms keygen, 158.58 ms signing, 313.88 ms verification, and 300 bytes peak heap RAM.

```

[2/3] ECDSA P-256 (secp256r1)
SIG-1 Key Generation Latency : 35262354 cycles | 146.93 ms
SIG-2 Public Key Wire Size : 65 bytes
SIG-3 Message Signing Latency : 38059479 cycles | 158.58 ms
SIG-4 Signature Wire Size : 71 bytes
SIG-5 Verification Latency : 75331369 cycles | 313.88 ms
SIG-6 Total Client Sign Time : 73321833 cycles | 305.51 ms
SIG-7 Peak Dynamic Heap Memory : 300 bytes

```

Figure 2: ESP32 Serial Output for ECDSA P-256 micro-benchmarks (100 runs @ 240 MHz).

5.3 Classical RSA-3072 Benchmark Execution Capture

Figure 3 displays the serial monitor output for RSA-3072, recording 9,630.80 ms key generation, 375.81 ms signing, 1.71 ms verification, and 2,068 bytes peak heap RAM.

```

[3/3] Classical RSA-3072 (PKCS#1 v1.5)
SIG-1 Key Generation Latency : 2311393015 cycles | 9630.80 ms
SIG-2 Public Key Wire Size : 384 bytes
SIG-3 Message Signing Latency : 90194354 cycles | 375.81 ms
SIG-4 Signature Wire Size : 384 bytes
SIG-5 Verification Latency : 410321 cycles | 1.71 ms
SIG-6 Total Client Sign Time : 2401587369 cycles | 10006.61 ms
SIG-7 Peak Dynamic Heap Memory : 2068 bytes

```

Figure 3: ESP32 Serial Output for Classical RSA-3072 micro-benchmarks (240 MHz).

5.4 Automated Benchmark Results Summary Table Capture

Figure 4 displays the automated console summary table printed at the conclusion of all micro-benchmarks.

BENCHMARK RESULTS SUMMARY				
KPI ID	Metric	Ed25519	ECDSA P-256	RSA-3072
SIG-1	Key Generation Time	8.56 ms	146.93 ms	9630.80 ms
SIG-2	Public Key Wire Size	32 B	65 B	384 B
SIG-3	Message Signing Time	9.42 ms	158.58 ms	375.81 ms
SIG-4	Signature Wire Size	64 B	71 B	384 B
SIG-5	Verification Time	19.80 ms	313.88 ms	1.71 ms
SIG-6	Total Client Sign Time	17.99 ms	305.51 ms	10006.61 ms
SIG-7	Peak Dynamic Heap RAM	108 B	300 B	2068 B

Figure 4: ESP32 Serial Output showing the automated Benchmark Results Summary Table.

6 Graphical Benchmark Visualizations

The figures below provide graphical comparisons across execution latencies, communication overheads, and dynamic memory footprints:

6.1 Execution Latency Comparison

Figure 5 compares key generation, signing, verification, and total client sign latency on a logarithmic scale.

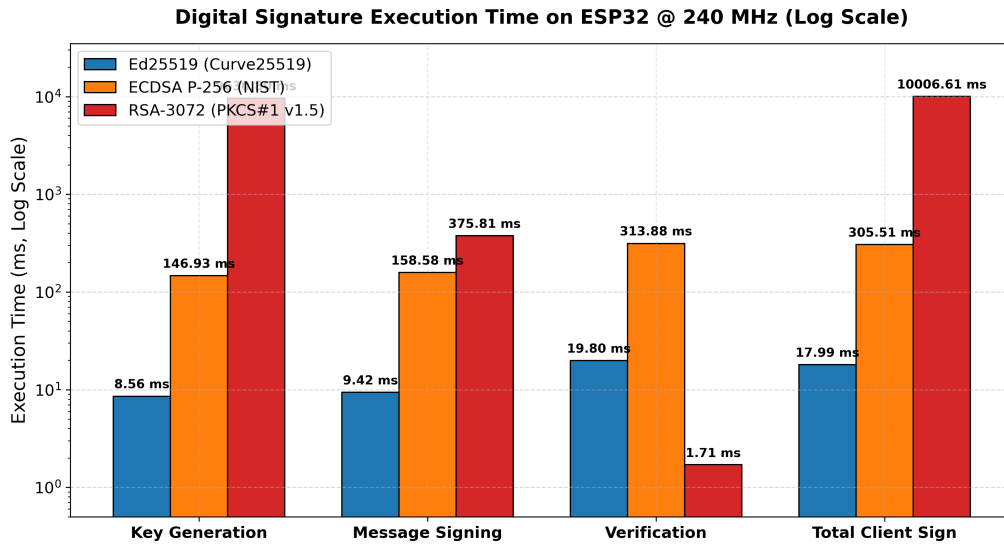


Figure 5: Execution latency comparison (logarithmic scale) for KeyGen, Signing, Verification, and Total Client Sign Time.

6.2 Communication Wire Overhead

Figure 6 breaks down the wire overhead required per mutual authentication exchange (Public Key + Signature size in bytes).

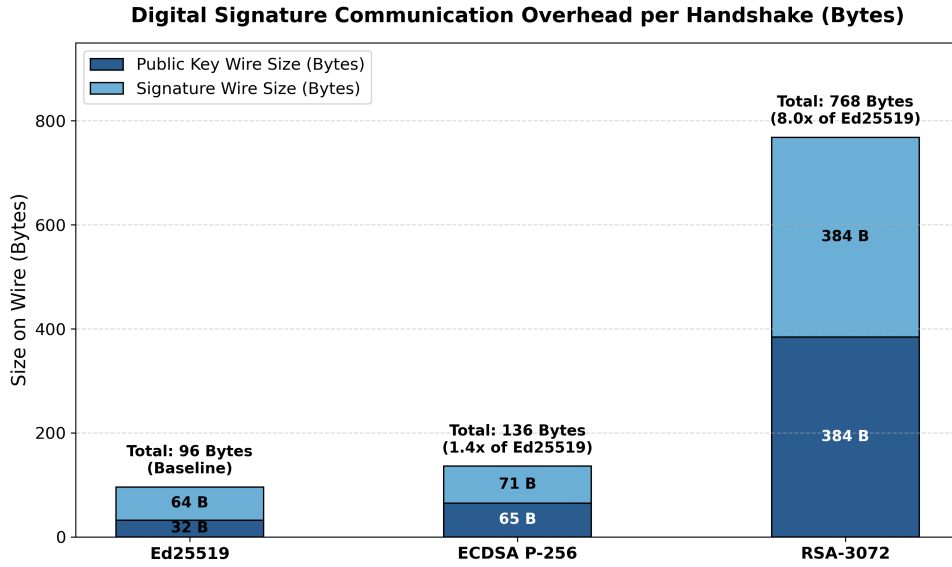


Figure 6: Digital signature communication wire overhead per authentication handshake (Bytes).

6.3 Peak Dynamic Heap RAM Overhead

Figure 7 compares the peak dynamic heap memory consumed during algorithm execution on the ESP32.

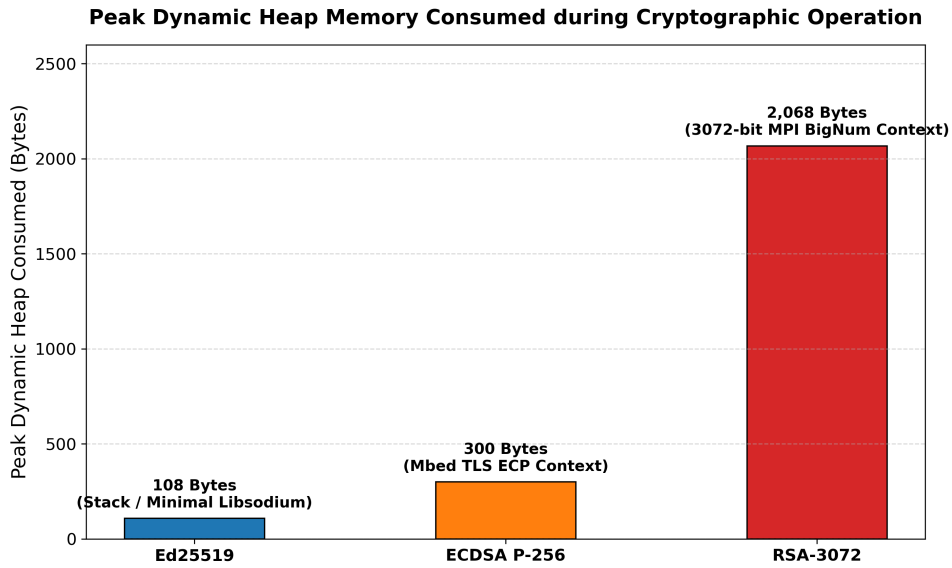


Figure 7: Peak dynamic heap memory allocation during digital signature execution on the ESP32.

7 Benchmarking Logic and Pseudocode

The pseudocode below details the exact measurement routines implemented on the ESP32 firmware, covering cycle-accurate timers, heap profiling, and test loops for all three algorithms:

7.1 Hardware Measurement Helpers

```
// Configuration: ESP32 runs at 240 MHz clock speed
#define FAST_RUNS      100
```

```

#define SLOW_RSA_RUNS    10
#define FAST_RSA_RUNS    100
#define MSG_SIZE         64
#define CPU_SPEED        240

// Direct read of the 32-bit hardware cycle register (ccount)
static inline uint32_t get_cycles() {
    uint32_t cycles;
    __asm__ __volatile__ ("rsr %0, ccount" : "=a"(cycles));
    return cycles;
}

// Convert raw CPU clock cycles into milliseconds
float cycles_to_ms(uint32_t cycles) {
    return (float)cycles / (CPU_SPEED * 1000.0f);
}

```

7.2 Ed25519 Micro-Benchmark Routine

```

// Target KPIs: SIG-1 (KeyGen), SIG-2 (Wire: 32 B), SIG-3 (Signing), SIG-4 (Sig: 64 B),
//              SIG-5 (Verify), SIG-6 (Total Client Sign), SIG-7 (Heap: 108 B)
uint8_t public_key[crypto_sign_PUBLICKEYBYTES]; // [SIG-2] 32 bytes
uint8_t private_key[crypto_sign_SECRETKEYBYTES];
uint8_t signature[crypto_sign_BYTES];           // [SIG-4] 64 bytes
unsigned long long sig_length = 0;

uint32_t heap_before = ESP.getFreeHeap();
uint32_t peak_heap = 0;

for (int i = 0; i < FAST_RUNS; i++) {
    // [SIG-1] Key Generation Latency (8.56 ms / 2,055,308 cycles)
    uint32_t start = get_cycles();
    crypto_sign_keypair(public_key, private_key);
    time_keygen[i] = get_cycles() - start;

    // [SIG-7] Peak Dynamic Heap Profiling (108 bytes)
    uint32_t used = heap_before - ESP.getFreeHeap();
    if (used > peak_heap) peak_heap = used;

    // [SIG-3] Message Signing Latency (9.42 ms / 2,261,378 cycles)
    start = get_cycles();
    crypto_sign_detached(signature, &sig_length, test_message, MSG_SIZE, private_key);
    time_sign[i] = get_cycles() - start;

    // [SIG-5] Verification Latency (19.80 ms / 4,750,949 cycles)
    start = get_cycles();
    crypto_sign_verify_detached(signature, test_message, MSG_SIZE, public_key);
    time_verify[i] = get_cycles() - start;

    // [SIG-6] Total Client Sign Time = SIG-1 + SIG-3 (17.99 ms / 4,316,686 cycles)
}

```

7.3 ECDSA P-256 Micro-Benchmark Routine

```

// Target KPIs: SIG-1 (KeyGen), SIG-2 (Wire: 65 B), SIG-3 (Signing), SIG-4 (Sig: 71 B),
//              SIG-5 (Verify), SIG-6 (Total Client Sign), SIG-7 (Heap: 300 B)
mbedtls_ecdsa_context main_key;
mbedtls_ecdsa_init(&main_key);
mbedtls_ecdsa_genkey(&main_key, MBEDTLS_ECP_DP_SECP256R1, mbedtls_ctr_drbg_random, &
    random_gen);

uint8_t signature[MBEDTLS_ECDSA_MAX_LEN];

```

```

size_t sig_length = 0;

uint32_t heap_before = ESP.getFreeHeap();
uint32_t peak_heap = 0;

for (int i = 0; i < FAST_RUNS; i++) {
    // [SIG-1] Key Generation Latency (146.93 ms / 35,262,354 cycles)
    mbedtls_ecdsa_context temp_key;
    mbedtls_ecdsa_init(&temp_key);
    uint32_t start = get_cycles();
    mbedtls_ecdsa_genkey(&temp_key, MBEDTLS_ECP_DP_SECP256R1, mbedtls_ctr_drbg_random, &
        random_gen);
    time_keygen[i] = get_cycles() - start;

    // [SIG-7] Peak Dynamic Heap Profiling (Captures ECP context allocation: 300 bytes)
    uint32_t used = heap_before - ESP.getFreeHeap();
    if (used > peak_heap) peak_heap = used;
    mbedtls_ecdsa_free(&temp_key);

    // [SIG-3] Message Signing Latency (158.58 ms / 38,059,479 cycles)
    // [SIG-4] Signature Wire Size (71 bytes DER sequence)
    sig_length = 0;
    start = get_cycles();
    mbedtls_ecdsa_write_signature(&main_key, MBEDTLS_MD_SHA256, message_hash, 32,
        signature, sizeof(signature), &sig_length,
        mbedtls_ctr_drbg_random, &random_gen);
    time_sign[i] = get_cycles() - start;

    // [SIG-5] Verification Latency (313.88 ms / 75,331,369 cycles)
    start = get_cycles();
    mbedtls_ecdsa_read_signature(&main_key, message_hash, 32, signature, sig_length);
    time_verify[i] = get_cycles() - start;

    // [SIG-6] Total Client Sign Time = SIG-1 + SIG-3 (305.51 ms / 73,321,833 cycles)
}

```

7.4 Classical RSA-3072 Micro-Benchmark Routine

```

// Target KPIs: SIG-1 (KeyGen), SIG-2 (Wire: 384 B), SIG-3 (Signing), SIG-4 (Sig: 384 B
// ),
// SIG-5 (Verify), SIG-6 (Total Client Sign), SIG-7 (Heap: 2068 B)
mbedtls_pk_context main_key;
mbedtls_pk_init(&main_key);
mbedtls_pk_setup(&main_key, mbedtls_pk_info_from_type(MBEDTLS_PK_RSA));
mbedtls_rsa_gen_key(mbedtls_pk_rsa(main_key), mbedtls_ctr_drbg_random, &random_gen,
    3072, 65537);

uint8_t signature[384]; // [SIG-4] 384 bytes
size_t sig_length = 0;

uint32_t heap_before = ESP.getFreeHeap();
uint32_t peak_heap = 0;

// [SIG-1] RSA Key Generation Loop (10 runs, 9,630.80 ms / 2,311,393,015 cycles)
for (int i = 0; i < SLOW_RSA_RUNS; i++) {
    mbedtls_pk_context temp_key;
    mbedtls_pk_init(&temp_key);
    mbedtls_pk_setup(&temp_key, mbedtls_pk_info_from_type(MBEDTLS_PK_RSA));

    uint32_t start = get_cycles();
    mbedtls_rsa_gen_key(mbedtls_pk_rsa(temp_key), mbedtls_ctr_drbg_random, &random_gen,
        3072, 65537);
    time_keygen[i] = get_cycles() - start;
}

```

```

// [SIG-7] Peak Dynamic Heap Profiling (Captures 3072-bit BigNum MPI context: 2,068
// bytes)
uint32_t used = heap_before - ESP.getFreeHeap();
if (used > peak_heap) peak_heap = used;

mbedtls_pk_free(&temp_key);
vTaskDelay(1); // Yield to prevent Watchdog Timeout
}

// [SIG-3] RSA Signing Loop (100 runs, 375.81 ms / 90,194,354 cycles)
for (int i = 0; i < FAST_RSA_RUNS; i++) {
    sig_length = 0;
    uint32_t start = get_cycles();
    mbedtls_pk_sign(&main_key, MBEDTLS_MD_SHA256, message_hash, 32,
                    signature, sizeof(signature), &sig_length,
                    mbedtls_ctr_drbg_random, &random_gen);
    time_sign[i] = get_cycles() - start;
    if (i % 10 == 0) vTaskDelay(1);
}

// [SIG-5] RSA Verification Loop (100 runs, 1.71 ms / 410,321 cycles)
for (int i = 0; i < FAST_RSA_RUNS; i++) {
    uint32_t start = get_cycles();
    mbedtls_pk_verify(&main_key, MBEDTLS_MD_SHA256, message_hash, 32, signature,
                      sig_length);
    time_verify[i] = get_cycles() - start;
}

// [SIG-6] Total Client Sign Time = SIG-1 + SIG-3 (10,006.61 ms / 2,401,587,369 cycles)

```